Category:

Web

Name:

Ping pwn

Message:

One of our junior network administrators created a simple web-based diagnostic tool to check the network connectivity from our server. He said it's safe because it has filtering functions to prevent potential attacks.

Can you check if it's safe or is vulnerable to potential attacks?

There should be flag.txt on the server for you!

URL: http://pingpwn.csg2025.local:5000 (from your virtual attack environment)

Hints:

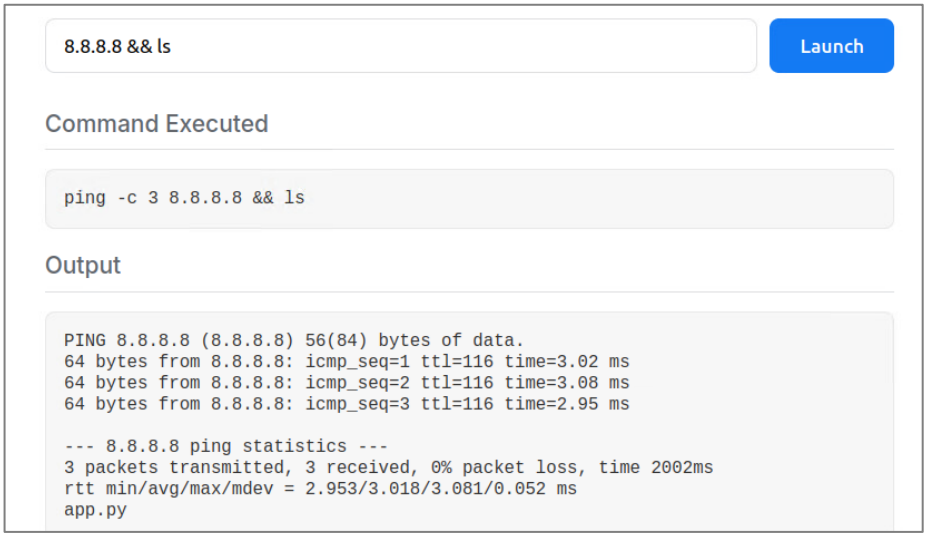
- The creator of this tool seems to have blocked the most famous command separator (;). A good step, but not enough!
- Successfully gained a shell, but remains a www-data user? Search for a file system for programs with elevated permissions!

Instructions:

1. Navigate to the provided link to see the simple "Ping Launcher" page with an input field. Testing with a typical IP address like 8.8.8.8 shows the output if ping command. This could suggest a potential OS Command Injection vulnerability.
2. Try to chain a simple command using a semicolon like *8.8.8.8; ls*. The "Command Executed" shows that the semicolon was removed. Let's now try to bypass the filters.

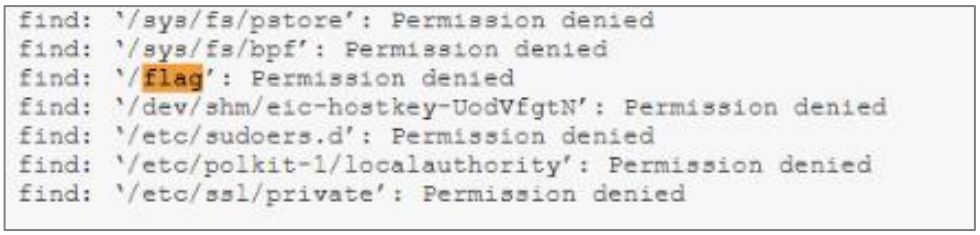


3. Since the semicolon is blacklisted, we need to find other command separators. A common alternative is the double ampersand &&, which executes the following command only if the previous one succeeds. Try *8.8.8.8 && ls* and see the output includes the web application file app.py. We now have command execution on the server.



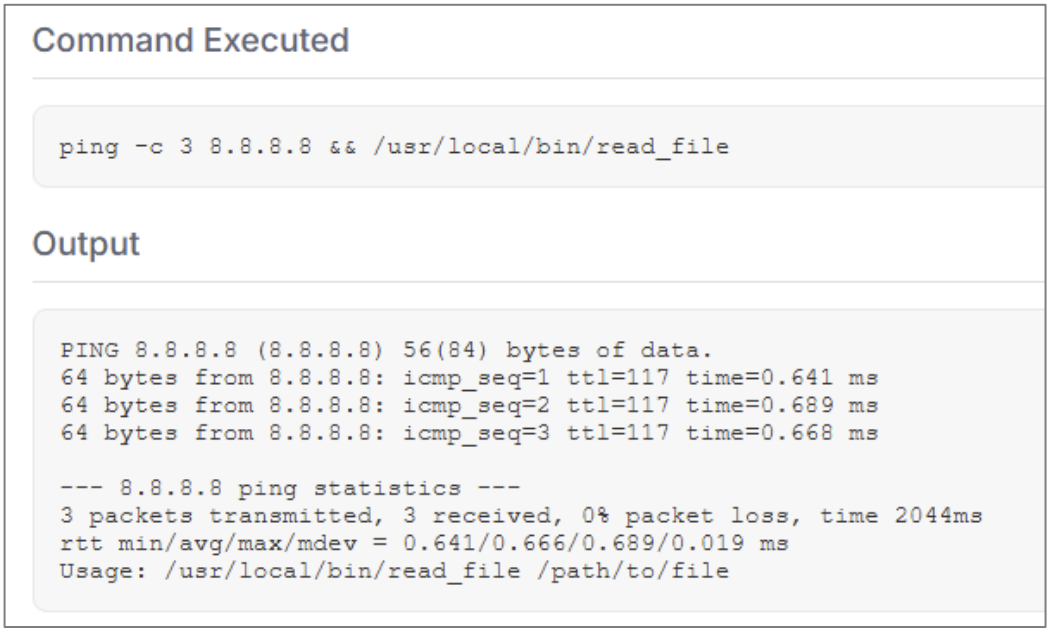
Similarly cat command is blacklisted. You will need to use alternative command, such as *tac*, to see the content of the file.

4. Now that we have a shell, let's find the flag.txt as specified in the challenge description. You can try this command *find / -name "flag.txt"*. Among the output, we can find a folder "flag" with a *Permission denied* message.

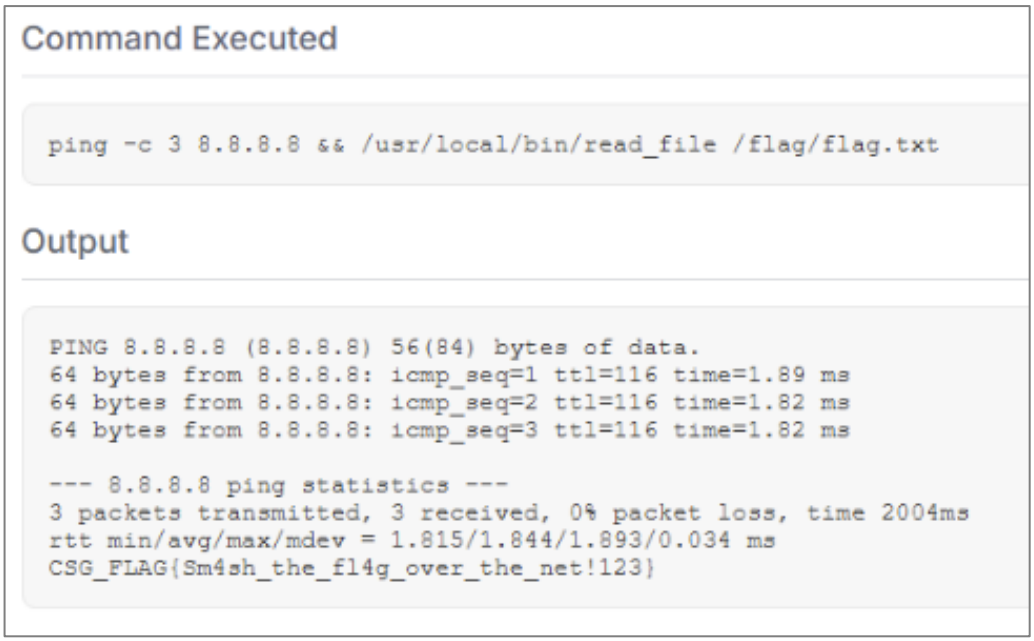


This is a big clue! It means there is flag.txt file under flag directory that the system doesn't want low-privilege users to see.

5. We need to escalate the privilege to root. A classic method for privilege escalation on Linux is to find misconfigured SUID binaries. There are programs that run with the permissions of the file owner, not the user who executes them. Let's search for the files with the SUID bit set *8.8.8.8 && find / -perm -4000*. In the output, among standard system utilities, one stands out as unusual: */usr/local/bin/read_file*. Since it's an SUID binary owned by root, if we execute it, it will run with root privileges. Let's use this program to our advantage. As is shown below, it requires a file as an argument.



6. Try *8.8.8.8 && /usr/local/bin/read_file /flag/flag.txt*. Because it has root privilege, the *Permission denied* error is now bypassed, and the content of the flag.txt is printed in the output!



Flag is:

CSG_FLAG{random_string}